Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Sözdizim

- Programlama dillerinin incelenmesi, doğal dillerin incelenmesi gibi, sözdizimi (syntax) ve anlambilim (semantics) incelemelerine ayrılabilir.
- Bir programlama dilinin **sözdizimi**, ifadelerinin, deyimlerinin (statements) ve program birimlerinin biçimidir. **Anlambilimi** ise bu ifade, deyim ve program birimlerinin taşıdığı anlamdır.
- Örneğin, Java dilinde bir *while* deyiminin sözdizimi şöyledir:

```
while (boolean_expr) statement
```

Anlambilimi

- Bu deyim biçiminin anlambilimi şöyledir: Boolean ifadenin mevcut değeri **true** ise, içteki deyim çalıştırılır. Ardından denetim örtük olarak yeniden Boolean ifadeye döner ve süreç tekrarlanır. Eğer Boolean ifade **false** ise, denetim while yapısından sonraki deyime aktarılır.
- Tartışma amacıyla genellikle ayrı ele alınsalar da sözdizimi (syntax) ile anlambilim (semantics) yakından ilişkilidir. İyi tasarlanmış bir programlama dilinde anlambilim, doğrudan sözdiziminden çıkarılabilmelidir; yani bir deyimin görünüşü, onun neyi gerçekleştirmeyi amaçladığını güçlü biçimde ima etmelidir.

Dili yakalama

- Genel olarak diller biçimsel olarak iki farklı şekilde tanımlanabilir: **tanıma (recognition)** ve **üretim (generation)** yoluyla (ancak bu yöntemlerin hiçbiri, tek başına, bir programlama dilini öğrenmek ya da kullanmak isteyenler için pratik bir tanım sağlamaz).
- Σ karakter alfabesini kullanan bir L dili olduğunu varsayalım. L'yi tanıma yöntemiyle biçimsel olarak tanımlamak için, Σ alfabesinden karakter dizilerini okuyabilen bir mekanizma R inşa etmemiz gerekir. Tanıma aygıtı (recognition device) adı verilen bu mekanizma, verilen bir karakter dizisinin L'ye ait olup olmadığını belirler.
- Başka bir deyişle, R ya girdiyi kabul eder ya da reddeder. Bu tür aygıtlar, doğru biçimlenmiş tümceleri hatalı olanlardan ayıran bir filtre gibi çalışır. Eğer R, Σ üzerindeki herhangi bir karakter dizisini yalnızca L'nin bir elemanıysa kabul ediyorsa, o zaman R, L'nin bir tanımıdır.

Dili yakalama

- Çoğu yararlı dil pratik olarak sonsuz olduğundan, bu süreç uzun ve verimsiz görünebilir. Ancak tanıma aygıtlarının amacı, bir dildeki tüm tümceleri listelemek değildir; farklı bir işlevleri vardır.
- Bir derleyicinin sözdizimsel analiz (syntax analysis) bölümü, derleyicinin çevirdiği dil için bir tanıyıcıdır. Bu rolde tanıyıcının, belirli bir karakter kümesinden oluşabilecek tüm olası dizileri test etmesi gerekmez. Yalnızca verilen programların dile ait olup olmadığını belirlemesi yeterlidir. Başka bir deyişle, sözdizimsel çözümleyici (syntax analyzer), verilen programların sözdizimsel olarak doğru olup olmadığını saptar.

Bağlam bağımsız gramerler

- Bir dili kabul eden söz dizim yapısının kurallarına gramer denir. Gramer dili analiz etmek için temel bileşendir. Burada bir gramer yapısı düzenli yada bağlam bağımsız olabilir. Bunun için aşağıdaki açıklamaya bakalım.
- 1950'lerin ortalarında, tanınmış dilbilimci Noam Chomsky, dört farklı dil sınıfını tanımlayan dört üretici aygıt (generative device) ya da gramer türü ortaya koymuştur (1956, 1959).
- Bu dilbilgisi sınıflarından ikisi—**bağlamdan bağımsız (context-free)** ve **düzenli (regular)** gramerleri—programlama dillerinin sözdizimini tanımlamada yararlı bulunmuştur.
- Programlama dillerindeki **token** biçimleri düzenli dilbilgileriyle tanımlanabilir. Programlama dillerinin tüm sözdizimi ise, küçük istisnalar dışında, bağlamdan bağımsız dilbilgileriyle ifade edilebilir.

Backus-Nour Form (BNF)

- **Gramer/Üstdil (metalanguage)**, başka bir dili tanımlamak için kullanılan dildir. Backus–Naur Form (BNF), programlama dillerini tanımlamak için kullanılan bir üstdildir.
- BNF, sözdizimsel yapılar için soyutlamalar kullanır. Örneğin basit bir Java atama deyimi <assign> soyutlaması ile gösterilebilir (soyutlama adlarını belirtmek için genellikle köşeli ayraçlar < > kullanılır).
- <assign> soyutlamasının tanımı şu şekilde verilebilir:

<assign> → <var> = <expression>

BNF – Basit bir dil

- Aşağıda BNF kural yapısı verilen dile ait örnekler türetiniz?

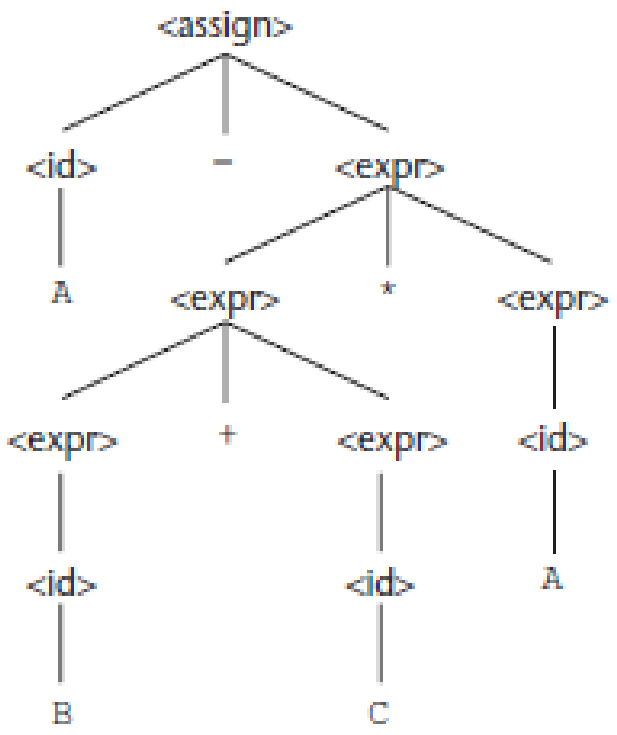
```
<program> → begin <stmt_list> end
<stmt_list> → <stmt>
            | <stmt> ; <stmt_list>
<stmt> → <var> = <expression>
<var> → A | B | C
<expression> → <var> + <var>
             | <var> – <var>
             | <var>
```

Sözdizim Ağaç Yapısı

Gramer kuralları

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <expr>
| <expr> * <expr>
| (<expr>)
| <id>

A = B * (A + C)

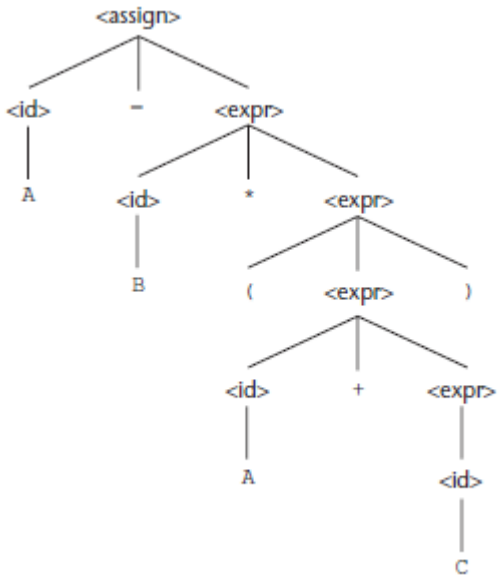


Sözdizim Ağaç Yapısı

Gramer kuralları

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <expr>
| <expr> * <expr>
| (<expr>)
| <id>

A = B * (A + C)



Belirsizlik

- Bir ifade iki farklı işlemci (operator) içerdiğinde, örneğin **x + y * z**, ortaya çıkan temel anlamsal sorunlardan biri işlem sırasıdır (örneğin bu ifadede önce toplama mı yapılır, yoksa çarpma mı?).
- Bu anlamsal soru, işlemcilere farklı **öncelik (precedence)** düzeyleri atanarak çözülebilir. Örneğin dil tasarımcısı tarafından * işlemcisine + işlemcisinden daha yüksek öncelik verilmişse, ifadede işlemcilerin yazılış sırasından bağımsız olarak önce çarpma işlemi gerçekleştirilir.
- Tartıştığımız basit ifadeler için, hem belirsizliği (ambiguity) ortadan kaldıran hem de + ve * işlemcileri arasında tutarlı bir öncelik ilişkisi belirleyen bir dilbilgisi (grammar) yazılabilir. Böylece işlemcilerin ifadede hangi sırayla göründüğüne bakılmaksızın doğru işlem sırası sağlanır.
- Doğru öncelik, farklı öncelik düzeylerine sahip işlemcilerin işlenenlerini (operand) temsil etmek için ayrı **nonterminal** semboller kullanılarak belirlenir. Bu yaklaşım, ek nonterminal'ler ve yeni kurallar gerektirir.

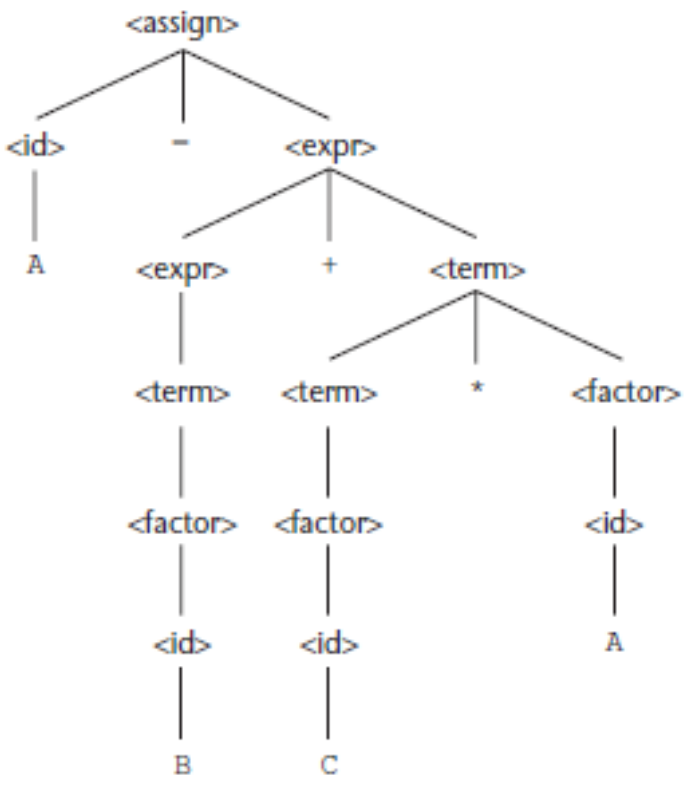
Belirsizlik

- Tartıştığımız basit ifadeler için, hem belirsizliği (ambiguity) ortadan kaldıran hem de + ve * işlemcileri arasında tutarlı bir öncelik ilişkisi belirleyen bir dilbilgisi (grammar) yazılabilir. Böylece işlemcilerin ifadede hangi sırayla göründüğüne bakılmaksızın doğru işlem sırası sağlanır.
- Doğru öncelik, farklı öncelik düzeylerine sahip işlemcilerin işlenenlerini (operand) temsil etmek için ayrı **nonterminal** semboller kullanılarak belirlenir. Bu yaklaşım, ek nonterminal'ler ve yeni kurallar gerektirir.

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

Belirsizlik

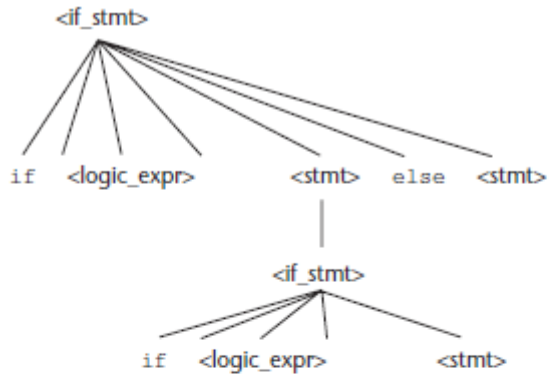
A = B + C * A

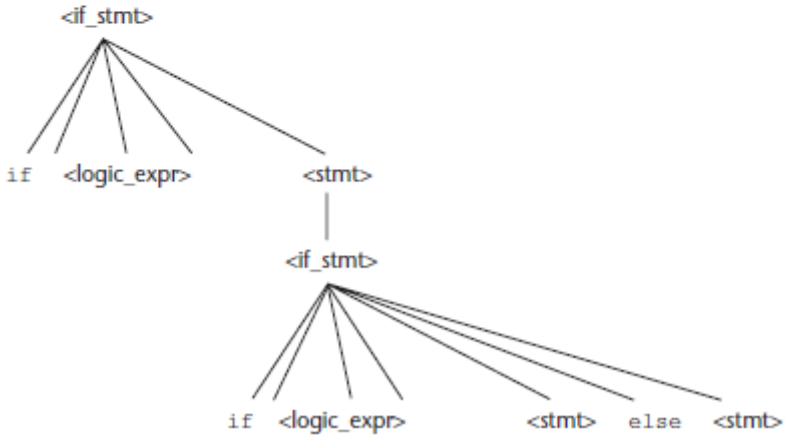


<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

If ifadesi

<stmt> → <matched> | <unmatched>
<matched> → **if** (<logic_expr>) <matched> **else** <matched>
| any non-if statement
<unmatched> → **if** (<logic_expr>) <stmt>
| **if** (<logic_expr>) <matched> **else** <unmatched>





Extended BNF (EBNF)

- BNF'deki bazı küçük kullanışsızlıklar nedeniyle bu gösterim çeşitli biçimlerde genişletilmiştir. Bu genişletilmiş sürümlerin çoğu **Extended BNF** ya da kısaca **EBNF** olarak adlandırılır; ancak hepsi birebir aynı değildir. Yapılan genişletmeler BNF'nin ifade gücünü artırmaz; yalnızca okunabilirliğini ve yazılabilirliğini iyileştirir.
- EBNF'nin farklı sürümlerinde genellikle üç temel genişletme bulunur. Bunlardan ilki, bir üretim kuralının sağ tarafındaki (RHS) **isteğe bağlı (optional)** kısmı göstermek için köşeli parantezlerin kullanılmasıdır.

- Örneğin C dilindeki bir if-else deyimi şu şekilde tanımlanabilir:

  <if_stmt> → if (<expression>)
  <statement> [else <statement>]

- Köşeli parantez kullanılmazsa, bu yapının sözdizimsel tanımı iki ayrı kuralla verilmek zorundadır:

  <if_stmt> → if (<expression>)
  <statement> | if (<expression>)
  <statement> else
  <statement>

Extended BNF

- İkinci genişletme, bir üretim kuralının sağ tarafında (RHS) süslü parantezlerin kullanılmasıdır. Süslü parantezler içine alınan bölümün **sıfır veya daha fazla kez tekrar edilebileceğini** (ya da tamamen atlanabileceğini) gösterir.
- Bu genişletme sayesinde listeler, özyineleme (recursion) ve iki ayrı kural kullanmak yerine tek bir kuralla tanımlanabilir.,
- Örneğin virgülle ayrılmış tanımlayıcı listeleri şu kuralla ifade edilebilir:

  <ident_list> → <identifier> {, <identifier>}

- Bu yapı, özyinelemenin yerine örtük bir yineleme (iteration) kullanır; süslü parantez içindeki bölüm istenildiği kadar tekrar edilebilir.
- Üçüncü yaygın genişletme, **çoktan seçmeli (multiple-choice)** seçeneklerle ilgilidir. Bir gruptan yalnızca bir öğe seçilecekse, seçenekler parantez içine alınır ve | (OR) işleci ile ayrılır.
- Örneğin:

  <term> → <term> (* | / | %) <factor>

- BNF'de bu <term> tanımı için üç ayrı kural yazmak gerekir:

  <term> → <term> * <factor> | <term> / <factor> | <term> % <factor>

- EBNF'de kullanılan köşeli parantezler [ ], süslü parantezler { } ve normal parantezler ( ) birer **metasembol**dür.
- Yani bunlar yalnızca gösterim amaçlı araçlardır; tanımlanan dilin terminal sembolleri değildirler. Eğer bu semboller tanımlanan dilde aynı zamanda terminal sembol olarak da kullanılıyorsa, karışıklığı önlemek için terminal olanlar altı çizili ya da tırnak içinde gösterilebilir.

BNF vs EBNF

BNF:

```
<expr> → <expr> + <term>
       | <expr> - <term>
       | <term>
<term> → <term> * <factor>
       | <term> / <factor>
       | <factor>
<factor> → <exp> ** <factor>
         <exp>
<exp> → (<expr>)
      | id
```

EBNF:

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
<factor> → <exp> { ** <exp>}
<exp> → (<expr>)
      | id
```

Attribute (Öznitelik) Gramerleri

- Öznitelik gramerinin (attribute grammars) **statik anlambilimi** tanımlamak için nasıl kullanılabileceğine dair basit bir örnek olarak, Ada'daki bir yordamın (procedure) sonunda yer alan ismin, yordamın başındaki isimle aynı olması kuralını ele alalım. (Bu kural BNF ile ifade edilemez.)
- <proc_name> nonterminal'inin **string** özniteliği (<proc_name>.string), derleyicinin procedure anahtar sözcüğünden hemen sonra okuduğu gerçek karakter dizisini temsil eder.
- Bir sözdizim kuralında aynı nonterminal birden fazla kez geçiyorsa, öznitelik gramerinde bunlar köşeli parantez içindeki alt indislerle ayırt edilir. Bu alt indisler ve köşeli parantezler tanımlanan dilin parçası değildir.

- Kural

<proc_def> → procedure <proc_name>[1]

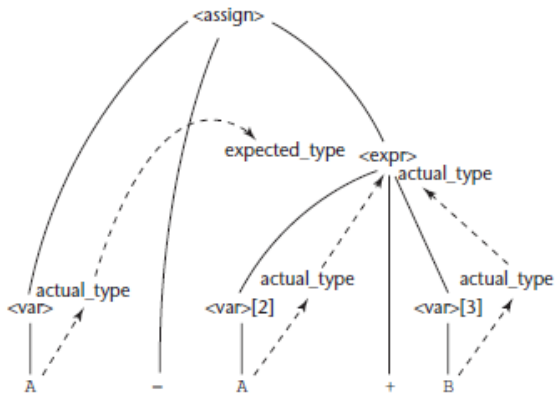
<proc_body> end <proc_name>[2];

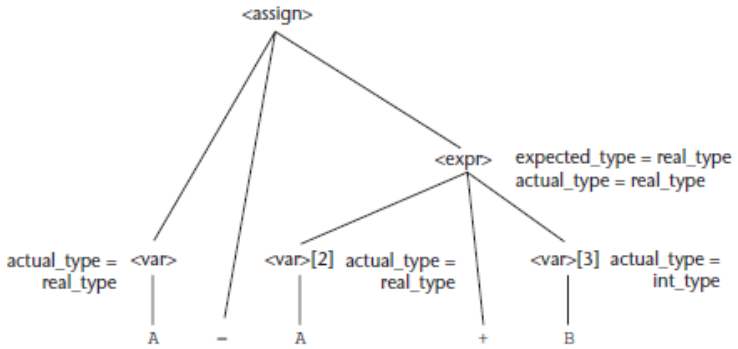
- Koşul

<proc_name>[1].string == <proc_name>[2].string

- Bu örnekte koşul, alt program başlığındaki <proc_name> nonterminal'inin string özniteliğinin, alt programın sonundaki end ifadesinden sonra gelen <proc_name> nonterminal'inin string özniteliğiyle aynı olması gerektiğini belirtir.

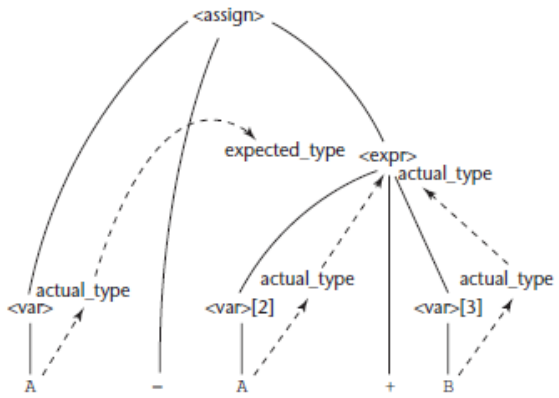
Attribute Gramer

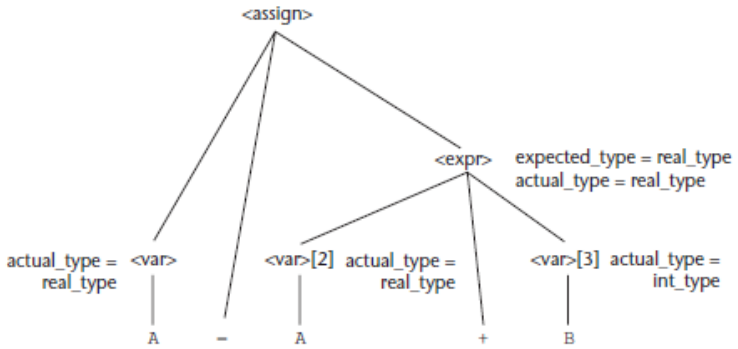




- Yandaki değişkenlerin altan ağacın üst kısımlarına doğru akışı verilmiştir.
- Burada indice ile belirlenen değerler yani sayı değerlerinin türü (int yada float) yukarı doğru iletilmektedir.

Attribute Gramer





1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

Anlambilim

- **Aksiyomatik anlambilim (axiomatic semantics)**, adından da anlaşılacağı gibi matematiksel mantığa dayanır ve anlambilim belirtiminin en soyut yaklaşımıdır.
- Bu yaklaşımda bir programın anlamı doğrudan tanımlanmaz. Bunun yerine, program hakkında **hangi özelliklerin ispatlanabileceği** tanımlanır.
- Başka bir ifadeyle, aksiyomatik anlambilim bir programın nasıl çalıştığını değil, programın doğru olduğunu nasıl kanıtlayabileceğimizi belirler.
- Anlambilim belirtimlerinin önemli kullanım amaçlarından biri **program doğruluğunu kanıtlamaktır**; aksiyomatik anlambilim de tam olarak bu amaca hizmet eder.
- Aksiyomatik anlambilimde makinenin ya da programın durumuna ve yürütme sırasındaki durum değişimlerine ilişkin bir model yoktur. Programın anlamı, her çalıştırmada değişmeyen değişkenler ve sabitler arasındaki ilişkilere dayanır.
- Bu yaklaşımın iki temel kullanım alanı vardır: **program doğrulama** ve **anlambilim belirtimi**. Burada odak program doğrulamadır.
- Aksiyomatik anlambilim, programların doğruluğunu kanıtlama amacıyla geliştirilmiştir. Böyle kanıtlar, programın belirtiminde tanımlanan hesaplamayı gerçekten gerçekleştirdiğini gösterir. Kanıtlarda her program ifadesinin önüne ve sonuna, program değişkenleri üzerindeki kısıtları belirten mantıksal ifadeler yazılır. Anlam, işlemsel anlambilimde olduğu gibi tüm makine durumuyla değil, bu mantıksal önermelerle tanımlanır.
- Kullanılan gösterim dili **yüklem mantığıdır (predicate calculus)**. Basit Boolean ifadeler çoğu zaman yeterli olsa da bazı durumlarda daha güçlü mantıksal ifadeler gerekir.
- Bir ifadenin anlamı, o ifadenin ilgili veriler üzerindeki doğruluk önermelerini nasıl değiştirdiğiyle tanımlanır.

Zayıf Koşul

- **En zayıf önkoşul (weakest precondition)**, ilgili sonkoşulun (postcondition) doğru olmasını garanti eden **en az kısıtlayıcı** önkoşuldur.
- Örneğin, belirli bir ifade için {x > 10}, {x > 50} ve {x > 1000} önkoşullarının hepsi geçerli olabilir. Ancak bunların en az kısıtlayıcısı, yani en zayıfı, **{x > 0}** önkoşuludur.
- Çünkü {x > 0} sağlandığında sonkoşul yine garanti edilir ve bu koşul diğerlerine göre daha geniş bir durum kümesini kapsar.
- **Çıkarım kuralı (inference rule)**, bazı önermelerin doğru olmasına dayanarak başka bir önermenin doğruluğunu çıkarsama yöntemidir. Genel biçimi şöyledir:

$$\frac{S_1, S_2, \ldots, S_n}{S}$$

- Bu yapı, S_1, S_2, …, S_n doğruysa S'nin de doğru olduğunun çıkarılabileceğini ifade eder. Üst kısım **öncül (antecedent)**, alt kısım ise **sonuç (consequent)** olarak adlandırılır.
- **Aksiyom (axiom)** ise doğruluğu varsayılan mantıksal bir ifadedir. Bu nedenle, öncülü olmayan bir çıkarım kuralı olarak düşünülebilir.
- Bazı program ifadeleri için, bir sonkoşuldan en zayıf önkoşulun hesaplanması basittir ve bir aksiyomla belirtilebilir. Ancak çoğu durumda bu ancak bir çıkarım kuralı ile ifade edilebilir.
- Bir programlama dilinde aksiyomatik anlambilimi kullanabilmek için—ister doğruluk kanıtı ister biçimsel anlambilim belirtimi amacıyla olsun—dildeki her ifade türü için bir aksiyom ya da çıkarım kuralı tanımlanmış olmalıdır.

Örnek

- Bir **atama ifadesinin (assignment statement)** anlamı, onun **önkoşulu (precondition)** ve **sonkoşulu (postcondition)** birlikte ele alınarak tanımlanır. Anlamı belirlemek için, verilen bir sonkoşuldan en zayıf önkoşulu hesaplayabilmemiz gerekir.
- Genel bir atama ifadesi x = E ve sonkoşulu Q olsun. En zayıf önkoşul P, şu aksiyomla tanımlanır:
- **$P = Q_{(x \rightarrow E)}$**
- Yani P, Q ifadesinde yer alan tüm x'lerin E ile değiştirilmesiyle elde edilir.
- Atama: a = b / 2 - 1
  Sonkoşul: {a < 10}
- Yerine koyma yapılır: b / 2 - 1 < 10
- Buradan: b < 22
- Dolayısıyla en zayıf önkoşul: {b < 22}
- Atama: x = 2 * y - 3
  Sonkoşul: {x > 25}
- Yerine koyma: 2 * y - 3 > 25
  Buradan: y > 14
- Sonuç olarak en zayıf önkoşul: {y > 14}
- Özetle, atama ifadelerinde en zayıf önkoşul hesaplama işlemi, sonkoşul üzerinde **sözdizimsel yerine koyma (substitution)** yapmaya dayanır.

Çıkarım

- Verilen ifade: $x = x + y - 3$ $\{x > 10\}$
- Atama aksiyomuna göre, sonkoşuldaki x yerine sağ taraf yazılır: $x + y - 3 > 10$
- Buradan: $y > 13 - x$
- Dolayısıyla en zayıf önkoşul:

  $\{y > 13 - x\}$

- Aksiyomatik anlambilim program doğruluğunu kanıtlamak için geliştirilmiştir.
- Bir ifade şu biçimde yazıldığında:
- **{P} S {Q}**
- bu, mantıksal bir önerme (teorem) olarak görülebilir. Eğer atama aksiyomu uygulanarak Q'dan elde edilen en zayıf önkoşul, verilen P ile aynıysa, önerme kanıtlanmış olur.

Çıkarım Örnek

- **{x > 3} x = x - 3 {x > 0}**
- Sonkoşulda yerine koyma yapılır:
- x - 3 > 0 → x > 3
- Elde edilen önkoşul, verilen önkoşulla aynıdır. Dolayısıyla önerme kanıtlanmıştır.

- **{x > 5} x = x - 3 {x > 0}**
- Aksiyomdan elde edilen önkoşul yine {x > 3} olur.
  Verilen önkoşul {x > 5} ise bundan daha güçlüdür.
- Burada **Sonuç Kuralı (Rule of Consequence)** kullanılır:

$$\frac{\{P\}\ S\ \{Q\},\ P' \Rightarrow P,\ Q \Rightarrow Q'}{\{P'\}S\{Q'\}}$$

- Eğer {P} S {Q} doğruysa,
  - P′ ⇒ P
  - Q ⇒ Q′
- ise

  {P′} S {Q′} çıkarımı yapılabilir.
- Bu kural şunu söyler: Önkoşul **güçlendirilebilir**, Sonkoşul **zayıflatılabilir**.
- Örnekte {x > 5} ⇒ {x > 3} olduğu için, ikinci önerme de geçerlidir.

$$\frac{\{x > 3\}x = x - 3\{x > 0\},\ (x > 5) \Rightarrow \{x > 3\},\ (x > 0) \Rightarrow (x > 0)}{\{x > 5\}x = x - 3\{x > 0\}}$$

Ardışık Çıkarım

$$\{P1\}\ S1\ \{P2\}$$
$$\{P2\}\ S2\ \{P3\}$$

$$\frac{\{P1\}\ S1\ \{P2\},\ \{P2\}\ S2\ \{P3\}}{\{P1\}\ S1, S2\ \{P3\}}$$

- y = 3 * x + 1;
- x = y + 3;
- {x < 10}

$$\{P3_{x2 \to E2}\}\ x2 = E2\ \{P3\}$$
$$\{(P3_{x2 \to E2})_{x1 \to E1}\}\ x1 = E1\ \{P3_{x2 \to E2}\}$$

- y < 7
- 3 * x + 1 < 7
- x < 2
- İfade dizsinin ön koşul olarak hesaplanır.

Koşul Seçimi

```
if x > 0 then
        y = y - 1
else
        y = y + 1
```

- y = y - 1 {y > 0} ifadesine atama aksiyomu uygulanırsa:
- y - 1 > 0 buradan y > 1 elde edilir. Bu, **then** dalının önkoşulu olarak kullanılabilir.
- Aynı şekilde **else** dalı için:
- y = y + 1 {y > 0}
- Sonkoşulda yerine koyma yapılır:
- y + 1 > 0 buradan y > -1 elde edilir.
- Şimdi elimizde iki önkoşul vardır:
- then dalı için: {y > 1}
- else dalı için: {y > -1}
- Çünkü {y > 1} ⇒ {y > -1}, yani {y > 1} daha güçlü bir koşuldur, **Sonuç Kuralı (rule of consequence)** sayesinde {y > 1} tüm seçim (if) ifadesi için ortak önkoşul olarak kullanılabilir.
- Böylece seçim yapısının tamamı için geçerli bir en zayıf önkoşul belirlenmiş olur.

Sorular

- Aşağıdaki gramer kurallarının belirsizlik yarattığını bir örnek ile gösteriniz?

  S → A

  A → A + A | id

  id → a | b | c

- Aşağıdaki ifadelerden hangileri verilen gramer kuralını kullanarak üretilmiştir.

  S → a S c B | A

  A → c A | c

  B> → d

  a. abcd
  b. acccbd
  c. acccbcc
  d. accd
  e. ccc

- Aşağıdaki ifadeler için zayıf önkoşulları hesaplayınız.

  a. a = 2 * (b - 1) - 1 {a > 0}
  b. b = (c + 10) / 3 {b > 6}
  c. a = a + 2 * b - 1 {a > 1}
  d. x = 2 * y + x - 1 {x > 11}